

Data Science

Feature Selection

Prof. Rayson Laroca
rayson@ppgia.pucpr.br

Avaliação Institucional

- Responda à **avaliação institucional** no endereço abaixo:
 - **Observação:** há uma questão de verificação de atenção!



<https://pucpr.vouavaliar.com.br/>

Institutional Evaluation

- Please complete the **institutional evaluation** at the address below:
 - **Caution:** this survey includes an attention check.



<https://pucpr.vouavaliar.com.br/>

Today's goals

- Understand why using all features is not always ideal
- Learn three main approaches for feature selection: filter, wrapper, and embedded methods

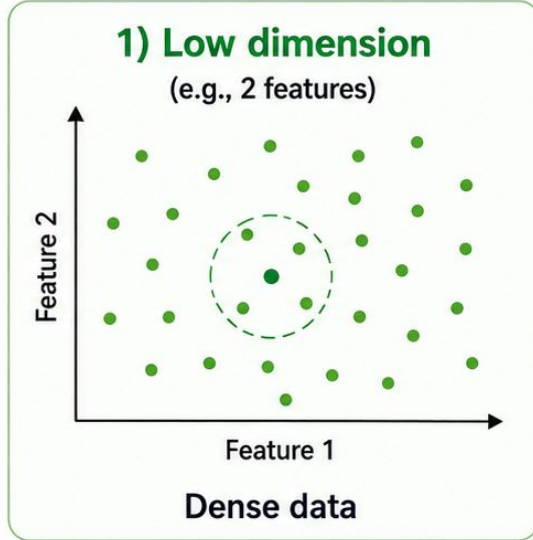
Why feature selection matters

- Supervised learning tasks, such as classification and regression, rely on **instances**, **features**, and **labels**
- **More** features require **more** labeled instances

The Curse of Dimensionality

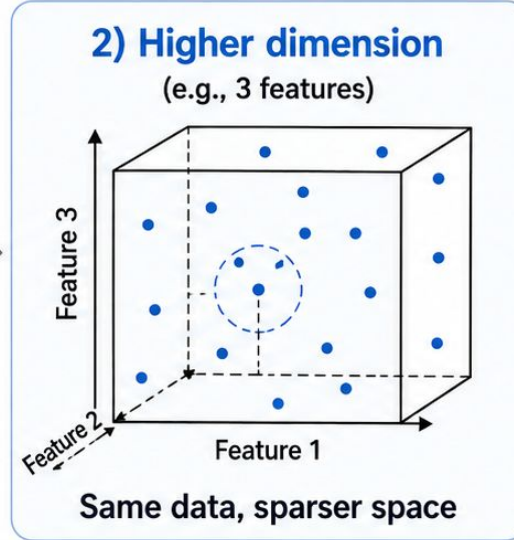
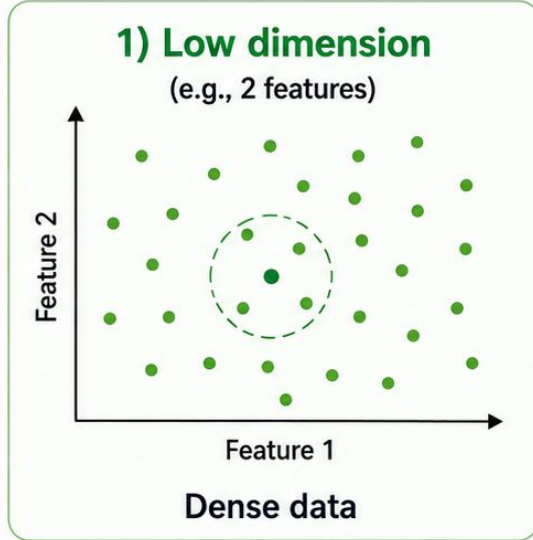
- When the dimensionality of a problem increases, the volume of the space grows so fast that the data available becomes sparse
- Assuming a feature space with 100 binary features (1.27×10^{30} combinations) and trillions of instances (e.g., 10^{12}), only about 10^{-18} of the feature space would actually be covered
- We need to decrease the feature space, either by **selecting features**, or by applying **dimensionality reduction**

The Curse of Dimensionality



More features → larger space

The Curse of Dimensionality



More features -> larger space

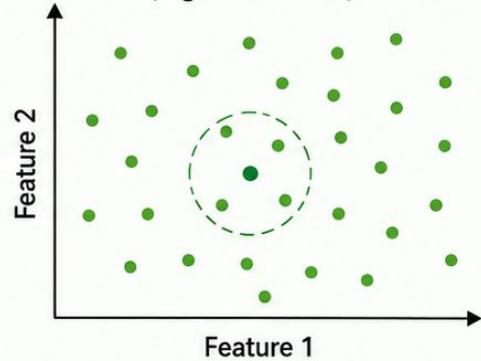


Same data -> sparser coverage

The Curse of Dimensionality

1) Low dimension

(e.g., 2 features)



Dense data

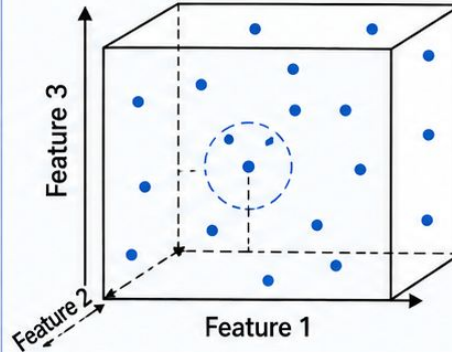


More features -> larger space



2) Higher dimension

(e.g., 3 features)



Same data, sparser space

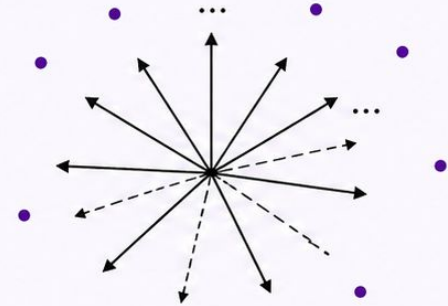


Same data -> sparser coverage



3) Very high dimension

(many features)



Most space is empty



Harder to find patterns

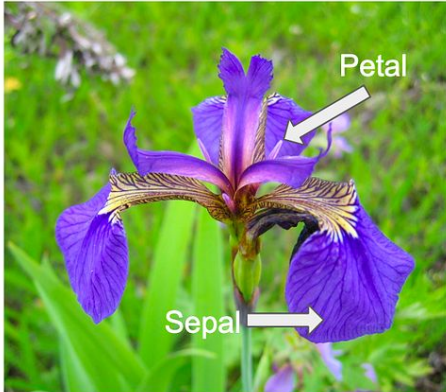
Motivation

- Using all available features does not necessarily lead to a more robust classifier
 - Irrelevant or redundant features can hinder the learning process and reduce model performance
- With few features, patterns may be easier to observe

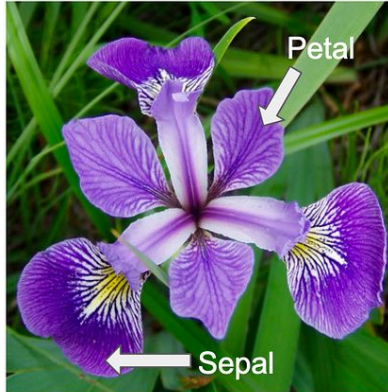
Example: Iris dataset

- Iris dataset (*not an image dataset; pictures below are only for illustration*):
 - 3 flower species (classes)

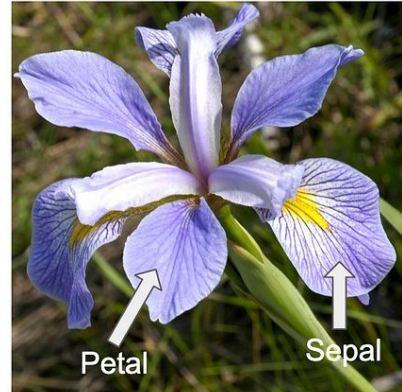
Iris setosa



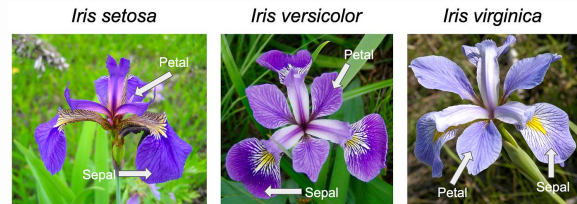
Iris versicolor



Iris virginica



Example: Iris dataset



- Iris dataset (*not an image dataset; pictures below are only for illustration*):
 - 3 flower species (classes)
 - 4 numerical features describing sepals and petals
 - sepal length, sepal width, petal length, petal width

Feature 1 (sepal.length)	Feature 2 (sepal.width)	Feature 3 (petal.length)	Feature 4 (petal.width)	Label (variety)
5.1	3.5	1.4	0.2	Setosa
7.0	3.2	4.7	1.4	Versicolor
6.3	3.3	6.0	2.5	Virginica

Practice

- Let's code a routine to test all possible feature subsets (as given in the next slide)
- We will compare a Decision Tree and a KNN classifier
- We will use Leave-One-Out validation for model evaluation

Subset size	Features				Results	
	SL	SW	PL	PW	DT	KNN
1 feature	X					
		X				
			X			
				X		
2 features	X	X				
	X		X			
	X			X		
		X	X			
		X		X		
			X	X		
3 features	X	X	X			
	X	X		X		
	X		X	X		
		X	X	X		
All features	X	X	X	X		

Subset size	Features				Results	
	SL	SW	PL	PW	DT	KNN
1 feature	X				62,67%	58,67%
		X			52,67%	48,00%
			X		94,00%	88,00%
				X	95,33%	88,00%
2 features	X	X			62,67%	70,67%
	X		X		94,00%	91,33%
	X			X	95,33%	92,67%
		X	X		94,00%	92,00%
		X		X	95,33%	92,67%
			X	X	95,33%	96,67%
3 features	X	X	X		94,00%	93,33%
	X	X		X	95,33%	94,00%
	X		X	X	95,33%	95,33%
		X	X	X	95,33%	95,33%
All features	X	X	X	X	95,33%	96,00%

- Is SW relevant?

- KNN with PL and PW:

- Simple model with the right features

Interpreting the results & Key discussion

- Some individual features are much more informative than others
 - Is sepal width (SW) relevant?
 - Does adding it improve performance?
 - Can it hurt performance?
- Petal features tend to perform better than sepal features
- A smaller subset may be enough
 - Should we keep a feature just because it is available?

Occam's Razor

- The law of parsimony: “the simplest explanation is most likely the right one”
- If you have two models with roughly the same accuracy, which one would you choose?
 - Go with the model with the least assumptions (features)!



William of Ockham (c. 1287–1347) was an English Franciscan friar and philosopher who produced significant works on logic, physics and theology.

Main Feature Selection Approaches

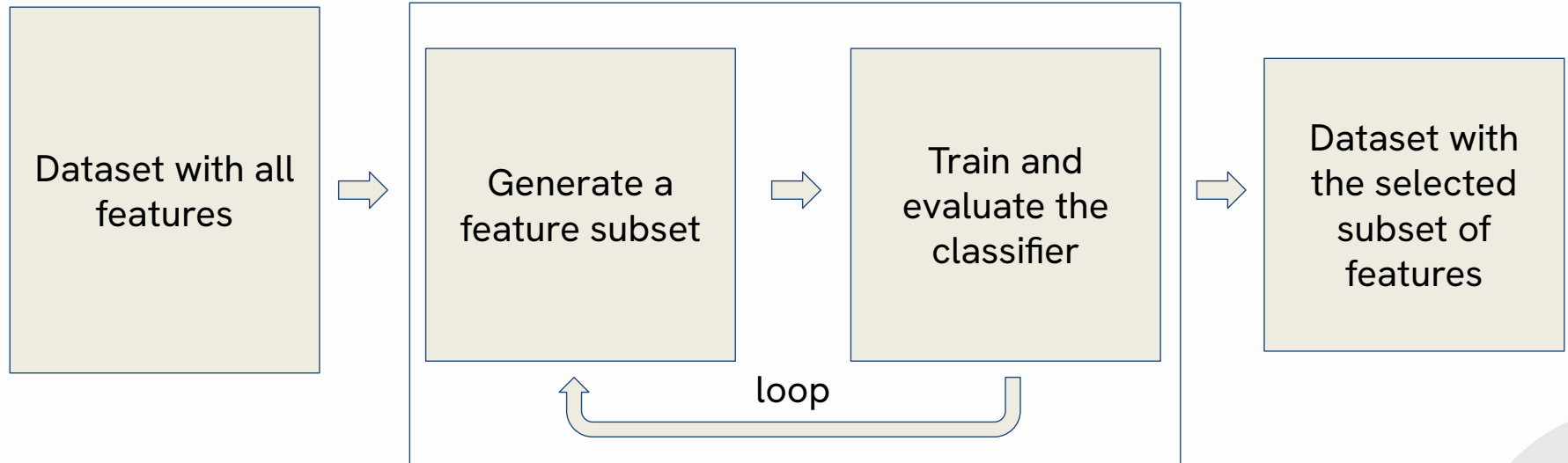
- **Wrapper** methods
- **Filter** methods
- **Embedded** methods
- Each approach uses a different strategy to decide which features matter

Wrapper methods

- Use a **specific model** to guide feature selection
- Evaluate feature subsets based on **model performance**
- In our first experiment, we tested every possible feature subset:
 - This is called **exhaustive search**
 - It can find the best subset for that model and evaluation setup
- However, exhaustive search does not scale well:
 - With d features, there are $2^d - 1$ non-empty feature subsets
 - This becomes impractical for datasets with many features

Wrapper – Recursive Feature Elimination (RFE)

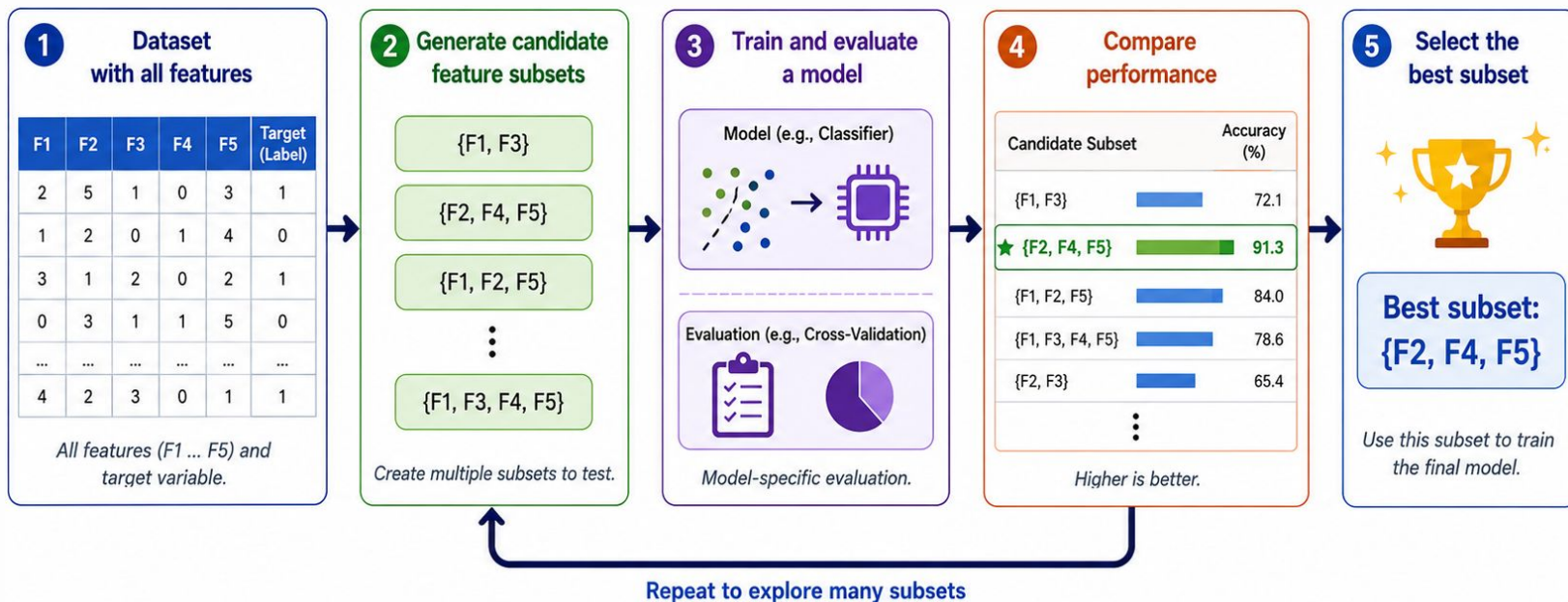
Recursive Feature Elimination



Practice

- Let's use **RFECV** (sklearn) to perform wrapper-based feature selection
- We will continue using the Decision Tree classifier and also test Logistic Regression
 - KNN cannot be used directly with RFE because it does not provide the feature-importance or coefficient values needed for recursive elimination

Wrapper methods



Uses a specific model to evaluate subsets



Can find strong feature combinations



Usually computationally expensive

Filter methods



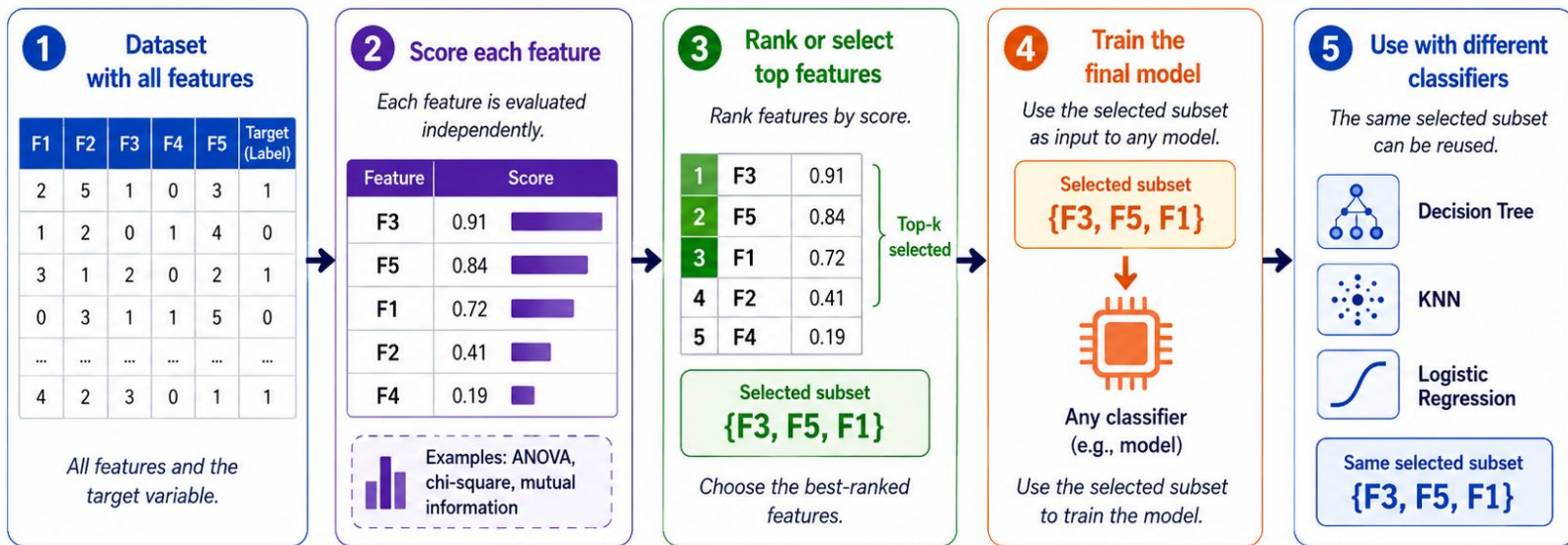
Filter methods

- Evaluate features **before** training the final model
- Usually much faster than wrapper methods
- **Classifier-independent**
- Produce either:
 - A ranking of feature relevance
 - A selected subset of the top features
- Useful as a simple and **efficient** first step for feature selection

Practice

- Let's use **SelectKBest** (sklearn) to apply filter-based feature selection

Filter methods



Filter methods are classifier-independent.



Fast and simple



Classifier-independent



Evaluates features individually



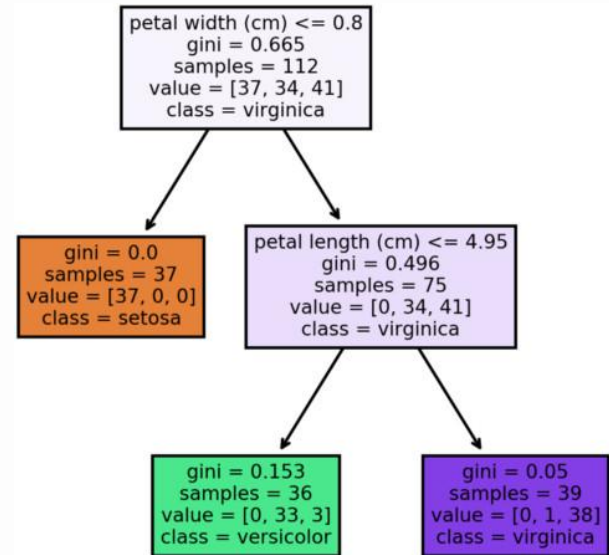
May miss useful feature interactions

Embedded methods

- Feature selection happens **during model training**
 - The model learns from the data while also identifying which features are most useful
- **Decision Trees** and **Random Forests** are common examples
 - Other techniques also exist, such as **L1-regularized models**
- **Takeaway:**
 - Feature selection is not a separate preprocessing step
 - It is part of how the model is trained

Decision tree-based feature selection

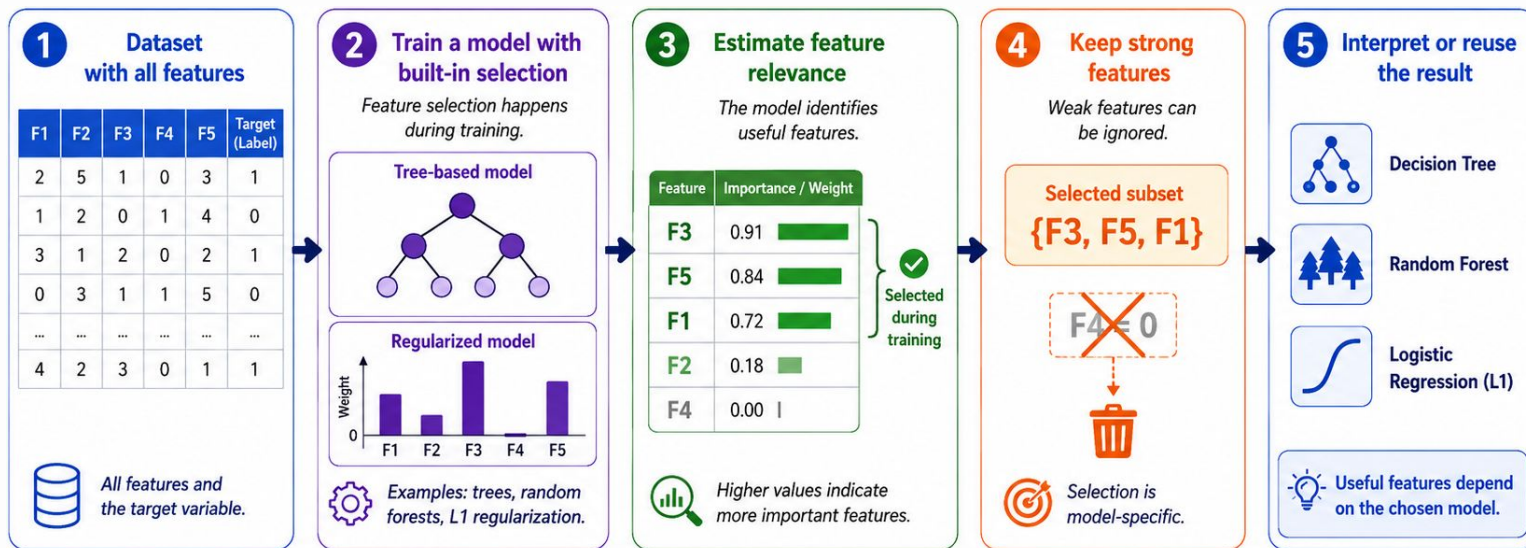
- You can create a decision tree model / random forest using your dataset
- Feature importance can be quantified
 - A tree chooses features that best split the data
 - Features used near the top are often very informative
- The tree can also be inspected visually



Practice

- Let's use Decision Tree and Random Forest models to estimate feature importance
- We will compare the rankings produced by both models:
 - Which features are considered most important?
 - Do both models highlight the same features?
- Let's also plot the Decision Tree to inspect how decisions are made internally

Embedded methods



Embedded methods perform feature selection during model training.



Integrated with training



More efficient than wrappers



Model-specific



Can use importances or coefficients

FINAL REMARKS



Common pitfall: data leakage

- Feature selection must be done **inside** the validation process
 - Selecting features before splitting the data can leak information from validation/test data
 - The test set must remain unseen!

Takeaways

- Feature selection can improve simplicity, speed, and interpretability
- Not every available feature is useful
- Wrapper methods are powerful but expensive
- Filter methods are fast and model-independent
- Embedded methods use the model training process itself

Project

- Use the remainder of today's class to work on your project
 - Review the features used in your dataset
 - Identify features that may be irrelevant or redundant
 - Test whether removing some features affects model performance
 - Compare at least these two settings:
 - Using all features
 - Using a selected subset of features
 - Be careful to avoid data leakage when selecting features